HealthCare+

Frontend-Only Project Plan
Healthcare Appointment Capacity & Queue Management

SPRINT 1 • HTML + CSS + VANILLA JAVASCRIPT

Team of 3 | Team : Vinod Manish Ajay | 10 approved screens

Decision	Frontend-only agreement
Goal	Build a polished, navigable prototype that follows the approved 10-screen reference.
Technologies	HTML5, CSS3 and vanilla JavaScript; no framework required.
Data	Fictional sample data in JavaScript; localStorage only if persistence is needed.
Backend	Not part of this plan. Do not build APIs, a database or authentication now.
Design	Blue, teal, green, navy, white and light blue-grey.
Deliverable	All 10 screens connected into one demo workflow.

1. Scope: Keep Sprint 1 Simple

Build only the frontend. The application should look and behave like a prototype using sample data in the browser. Focus on consistent UI, navigation and simple interactions—not production architecture.

- Use HTML for structure, CSS for styling and JavaScript for interactions.
- Use fictional doctors, patients, appointments, schedules and queue entries.
- Make search, filters, date/slot selection, form validation, confirmation, check-in and queue buttons work in the demo.
- Use one shared sample-data structure so the three members' screens can be integrated.
- Use localStorage only if you need demo data to survive a page refresh in the same browser.
- Do not create a backend, API, database, login system or real-time service in this sprint.

2. The 10 Approved Screens and Ownership

Screen	Owner	Layout/components	Expected behavior
1. Patient Home / Find a Doctor	Member 2	White navbar; hero heading; doctor/department search; department cards; optional doctor results.	Search filters sample doctors/departments. Selecting a doctor opens Screen 2.
2. Doctor Details & Availability	Member 2	Doctor profile, specialty, fee/duration sample, date selector, time-slot grid and Continue button.	Select one available slot. Full or blocked slots cannot be selected. Pass doctor/date/time to Screen 3.
3. Patient Appointment Booking	Member 2	Patient form on left; appointment summary on right; Confirm Booking button.	Validate required name and phone fields. Create a demo appointment ID and move to Screen 4.
4. Booking Confirmation	Member 2	Success message, appointment ID, doctor/date/time/status and check-in instructions.	Show details from the booking. Say token is assigned at reception; do not invent one before check-in.
5. Admin Dashboard	Vinod	Staff sidebar, top header, four KPI cards, appointment summary and department capacity bars.	Display figures derived from sample data, not unrelated hardcoded values.
6. Appointment Management	Vinod	Search/filter toolbar, appointment table, view-details panel/modal.	Search by patient or ID; filter by date/status; allow demo cancellation/rescheduling only if time permits.
7. Doctor Management	Vinod	Doctor list/table, search, Add/Edit Doctor form or modal.	Add/edit sample doctors, search and change active status.
8. Schedule / Capacity Management	Member 3	Doctor/date selectors, total/booked/available cards, slot grid and legend.	Show slot capacity and booked count; allow demo block/unblock action.
9. Receptionist Patient Check-in	Member 3	Appointment search, appointment detail card, Check-in button; optional walk-in tab.	Find a demo appointment, assign one token and update its status. Walk-ins are optional for Sprint 1.
10. Live Queue Management	Member 3	Department/doctor selectors, Now Serving, Next Patient, waiting count and queue table.	Call next, start consultation, complete or skip a patient using valid status transitions.

3. Visual Design System

Token	Hex	Use
Primary blue	#2563EB	Primary buttons, links and selected controls
Teal	#0D9488	Secondary actions and active navigation
Healthcare green	#16A34A	Confirmed bookings and successful actions
Dark navy	#123047	Staff sidebar and headings
Light background	#F3F8FA	Main page background
White	#FFFFFF	Cards, forms and tables
Main text	#24352E	Body and table text
Border	#DCE6E9	Card outlines and dividers
Warning	#D97706	Waiting and partially booked
Danger	#DC2626	Cancelled, full or validation error

Layout rules

- Font: Inter with Arial/sans-serif fallback.
- Staff pages use the same dark navy sidebar and white top header.
- Patient pages use a white horizontal navbar; do not use the staff sidebar on patient pages.
- Use light blue-grey page background and white cards with subtle borders.
- Card radius: 10–12px; button radius: 8px; card padding: 16–20px.
- Suggested desktop content width: 1200px; page spacing: 24px desktop and 16px mobile.
- Suggested input height: 42–44px; body text: 14–16px; page heading: 24–28px.
- Use a responsive breakpoint around 768px. Tables may scroll horizontally on small screens.
- Every status badge must contain text as well as color. Keep keyboard focus visible.

4. Recommended Folder Structure

```
healthcare-appointment-queue-management/
├── frontend/
│   ├── index.html
│   ├── pages/
│   │   ├── patient/
│   │   │   ├── home.html
│   │   │   ├── doctor-details.html
│   │   │   ├── booking.html
│   │   │   └── confirmation.html
│   │   ├── admin/
│   │   │   ├── dashboard.html
│   │   │   ├── appointments.html
│   │   │   └── doctors.html
│   │   └── reception/
│   │       ├── schedules.html
│   │       ├── check-in.html
│   │       └── queue.html
│   ├── css/
│   │   ├── global.css
│   │   ├── components.css
│   │   ├── patient.css
│   │   ├── admin.css
│   │   ├── reception.css
│   │   └── responsive.css
│   ├── js/
│   │   ├── main.js
│   │   ├── mock-data.js
│   │   ├── shared-state.js
│   │   ├── utils.js
│   │   ├── patient.js
│   │   ├── admin.js
│   │   └── reception.js
│   └── assets/
│       ├── images/
│       └── icons/
├── docs/
│   ├── requirements.md
│   ├── ui-guidelines.md
│   └── sprint-plan.md
├── .github/
│   └── pull_request_template.md
├── .gitignore
├── CONTRIBUTING.md
└── README.md
```

Keep this structure deliberately simple. The JavaScript files are grouped by responsibility, but there is no module/build system required. Create files when you need them rather than committing lots of empty files. Use relative links and test the pages through VS Code Live Server.

Path	Purpose
pages/patient/	Four patient-facing HTML screens
pages/admin/	Three administration HTML screens
pages/reception/	Three reception/queue HTML screens
global.css	CSS variables, reset and base styles
components.css	Reusable cards, buttons, forms, tables and status badges
responsive.css	Shared tablet/mobile rules
mock-data.js	Fictional doctors, departments, slots and appointments
shared-state.js	One browser-side state object shared by page scripts
utils.js	ID generation, date formatting and small helpers
patient.js / admin.js / reception.js	Interactions for each screen group

5. Starter CSS: Shared Colors and Components

```
/* frontend/css/global.css */
:root {
  --primary: #2563eb;
  --teal: #0d9488;
  --success: #16a34a;
  --navy: #123047;
  --background: #f3f8fa;
  --surface: #ffffff;
  --text: #24352e;
  --muted: #64748b;
  --border: #dce6e9;
  --warning: #d97706;
  --danger: #dc2626;
  --radius: 10px;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Inter, Arial, sans-serif;
  color: var(--text);
  background: var(--background);
  font-size: 15px;
  line-height: 1.5;
}

button, input, select, textarea { font: inherit; }
button { cursor: pointer; }
button:disabled { cursor: not-allowed; opacity: .55; }

.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 20px;
}

.btn-primary {
  background: var(--primary);
```

```
  color: white;
  border: 0;
  border-radius: 8px;
  padding: 11px 16px;
}

.btn-secondary {
  background: white;
  color: var(--teal);
  border: 1px solid var(--teal);
  border-radius: 8px;
  padding: 10px 16px;
}

:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 2px;
}
```

Each member should extend the shared styles instead of creating their own color palette.

6. Simple Sample Data and State

Use consistent IDs, date format and status values. Do not build a database for this sprint.

```
// frontend/js/mock-data.js
const doctors = [
  {
    id: "DOC001",
    name: "Dr. Ananya Rao",
    department: "General Medicine",
    consultationMinutes: 15,
    status: "active"
  }
];

const appointments = [
  {
    id: "APT1001",
    patientName: "Demo Patient",
    phone: "9000000000",
    doctorId: "DOC001",
    department: "General Medicine",
    date: "2026-10-26",
    time: "10:00",
    status: "confirmed",
    token: null
  }
];
```

Keep statuses consistent

For the simple demo, use: confirmed, checked_in, waiting, called, in_consultation, completed, cancelled and skipped.

Simple frontend rules

- Do not let a user select a full or blocked slot.
- Do not allow a booking if the selected slot has reached its sample capacity.
- Do not generate another token if the same appointment is checked in twice.
- Only show queue actions that make sense for the current status.

- Update dashboard counts from the same sample data/state wherever practical.
- If using localStorage, explain that data is saved only in that browser and is not shared between users.

7. Team Responsibilities

Member	Screens/files	Responsibilities
Vinod — Lead	Screens 5–7; global.css; components.css; main.js; mock-data coordination; README/docs	Create repo, establish UI rules, build admin screens, review PRs and integrate the workflow.
Member 2 — Patient	Screens 1–4; patient.css; patient.js	Build search, doctor details, date/time selection, booking form and confirmation.
Member 3 — Reception	Screens 8–10; reception.css; reception.js	Build schedule/capacity, check-in, token generation and queue controls.

Team rules

- Do not independently change shared colors, global CSS or data field names.
- Tell the lead before changing a shared file.
- Each member tests their own screens before requesting review.
- Use only fictional data and do not commit secrets or patient details.
- Ask for help early if navigation or shared-state integration is blocked.

8. GitHub Setup and Daily Workflow

1. Create a private GitHub repository named healthcare-appointment-queue-management.
2. Invite the two teammates from repository Settings → Collaborators.
3. Create the starter folder structure, global CSS and README on a setup branch.
4. Open a pull request into main and merge the approved foundation.
5. Each member creates their feature branch from the latest main.

```
git clone https://github.com/YOUR_USERNAME/healthcare-appointment-queue-management.git
cd healthcare-appointment-queue-management
git switch -c chore/frontend-foundation

# After creating starter files:
git add .
git commit -m "chore: add frontend structure and shared styles"
git push -u origin chore/frontend-foundation

# Open a pull request on GitHub.
```

Feature branch names

Branch	Owner
feature/admin-ui	Vinod
feature/patient-ui	Member 2
feature/reception-ui	Member 3

After completing a task

```
git add .
git commit -m "feat: add patient booking screen"
git push -u origin feature/patient-ui
# Open a pull request, describe what changed and how you tested it.
```

Use commit prefixes: feat (feature), fix (bug), style (visual), docs (documentation), chore (setup) and test (tests). Avoid pushing feature work directly to main.

9. Suggested Seven-Day Work Plan

Day	All-team target
1	Create repo, folder structure, shared colors, starter pages and agree on data names.
2	Build initial layouts: patient home/details, admin dashboard and schedule.
3	Finish static layouts for all 10 screens.
4	Add search, slot selection, forms, filters and queue buttons.
5	Connect booking, appointment list, check-in, token and queue using shared sample state.
6	Test invalid inputs, full slots, duplicate check-in, statuses and mobile layout.
7	Review/merge pull requests, fix major defects, update README and prepare demo.

This is a suggested schedule; adjust it to the team's actual availability.

10. End-to-End Demo Flow

6. Patient searches for a doctor on Screen 1.
7. Patient selects a date and available slot on Screen 2.
8. Patient enters demo details and confirms booking on Screen 3.
9. Screen 4 shows the appointment ID and confirmation details.
10. The same appointment appears in Screen 6's appointment list.
11. Reception finds and checks in the appointment on Screen 9.
12. A token is assigned and the queue updates on Screen 10.
13. Reception calls the next patient and changes the queue status.
14. Dashboard and capacity values reflect the sample state where connected.

11. Test Checklist and Definition of Done

Test	Expected result
Search for a doctor	Matching results or a helpful empty state appears.
Choose a full slot	Slot is disabled or booking is rejected.
Submit empty form	Required field errors appear.
Book a valid slot	Demo appointment is created with an ID.
Check in the same appointment twice	No duplicate active token is generated.
Call next patient	A valid waiting patient is selected.
Complete a consultation	Status changes to completed; patient is not called again.
Open on mobile	Layout remains usable with no major overflow.
Navigate between pages	All links work and selected booking details are preserved.

Sprint 1 is complete when

- All 10 approved screens exist and are navigable.
- All screens use the same blue/teal/green theme and shared components.
- Search, slot selection, form validation and confirmation work.
- Reception check-in generates a token and updates the demo queue.
- The team has tested desktop and mobile sizes.
- No major broken links or console errors remain.
- All approved code is merged into main and README has run instructions and screenshots.

12. README and Documentation Requirements

File	Include
README.md	Project overview, screenshots, tech stack, how to run, screen list, team members and limitations.
CONTRIBUTING.md	Branch naming, commits, PR process and coding rules.
docs/requirements.md	Project goals and the 10-screen scope.
docs/ui-guidelines.md	Color palette, typography, layout and reusable components.
docs/sprint-plan.md	Assignments, tasks, dates and acceptance criteria.

13. Important Limitations of the Frontend Demo

- This is a browser prototype, not a production hospital system.
- JavaScript checks can be bypassed; capacity rules are for demonstration only.
- localStorage is limited to the same browser profile and does not synchronize between team members or devices.
- Do not use real patient information or describe the demo as secure or compliant for clinical use.
- Backend, database, authentication and advanced optimization are explicitly deferred; do not spend Sprint 1 time building them.

14. Team Lead's Immediate Checklist

☐ Create the repository and invite both teammates.

☐ Commit the folder structure, global CSS and entry page.

☐ Agree on sample data fields and appointment statuses.

☐ Create GitHub issues for each screen and assign owners.

☐ Build Screens 5–7 and review the common styles.

☐ Integrate the patient and reception modules before the final day.

☐ Run the full demo flow and update README.

Final Agreement

For this phase, the team will build only the 10-screen frontend using HTML, CSS, vanilla JavaScript and fictional sample data. No backend or advanced features are required. Keep the UI consistent, make the core demo flow work, and leave future expansion for a later phase.